

Chapter 3 Critical Thinking

Questions 1–5 and 11 • Textbook page 70

1. Identifier names

- a) Four legal identifiers are numberOfStudents, gradeAverage, firstName, and totalCost.
- b) 2ndGrade is illegal because it starts with a number. class is illegal because it is a Java keyword. grade average is illegal because it contains a space. total-cost is illegal because a hyphen is not allowed in an identifier.

2. Declaring numBeads

- a) In two statements: `int numBeads; numBeads = 5;`
- b) In one statement: `int numBeads = 5;`

3. Final value of yourNumber

- a) The final value is 13. yourNumber starts at 4, so myNumber becomes 8. Then 5 is added to 8.
- b) The final value is 11. myNumber receives $4 + 7$, and that value is copied into yourNumber.

4. Appropriate data types

Value	Type	Reason
Number of basketballs in a store	int	It is a whole-number count.
Price of a basketball	double	A price can include a decimal portion.
Number of players on a team	int	It is a whole-number count.
Average age of the players	double	An average can include a decimal portion.
Whether a player received a jersey	boolean	The answer is true or false.
First initial of a player's first name	char	Only one character is needed.

5. Primitive types, abstract data types, classes, and objects

- a) A primitive type is built into Java and stores one basic value, such as int, double, char, or boolean. An abstract data type combines data with the operations that work on that data and is usually represented by a class.
- b) A class is the blueprint that defines data and methods. An object is one actual instance made from that class.

11. Type casting and rounding

Given: `int j = 5; double k = 1.6; int y; double z;`

- a) `y = (int)(j * k + 0.5);`** The multiplication produces a double, so it is rounded and cast before being stored in `y`.
- b) `z = (double)j * k;`** Java would promote `j` automatically, but the cast clearly shows that real-number multiplication is intended.
- c) `z = k * k;`** No cast is needed because both operands and the destination are double.
- d) `j = (int)(k + 0.5);`** The double value is rounded and cast before being stored in the `int` variable.
- e) `k = j;`** No cast is needed because Java can automatically promote an `int` to a double.
- f) `y = j + 3;`** No cast is needed because the operands and destination are all `int` values.